

Sessional - III

DATE: _____

PAGE: _____

1950-60

Perceptron

→ Prob: Only works with linearly separable

1970-80

MLFF BPN

→ Back Propagation Network

1980-90

SVM

← Optimization Problem into a

2005-7

CNN

Quadratic programming prob.

→ Scalable

Problem: gives locally minimum solution.

Prob:

Increase Dimension for non linear functions, which means higher learning.

① Is the three layer BPN solve any non-linear prob? → Yes. (I/P layer, Hidden layer, O/P layer)

If we increase Hidden layer, there won't be any change in output. only learning will increase. (High Computation)

$$y = f(x) \leftarrow \text{hidden}$$

$$y = (f'(f''(x)))$$

② How many neurons in each layer?

→ Handwritten digit recognition

* Output layer: - 10 cone for each digit)

- 4 $\begin{pmatrix} 0000 \rightarrow 0 \\ 0001 \rightarrow 1 \\ 0010 \rightarrow 2 \end{pmatrix}$

* Input layer

= No. of features

(16x16) Binary Image of handwritten digit
= (256 input neurons)

Reduce learning time

Feature engineering can help to reduce # of Inform Break down image into these categories:

- line, Horizontal, vertical.
- curve, circle
- space

- 3 features:
#hline
#vline #curve

* Hidden layer :

↳ Unknown non-linear function.

should be <input & > output.

We can avg out: $\frac{M+N}{2}$ where $M = \# \text{ of i/p layer}$

or $\frac{M \times N}{2} \rightarrow$ space will be increase

$N = \# \text{ of o/p layer}$

→ How to train your ANN ?

Eg. Digit Recognition

Data set: <Handwritten digit> <Digit>

A

A

B

B

C

C

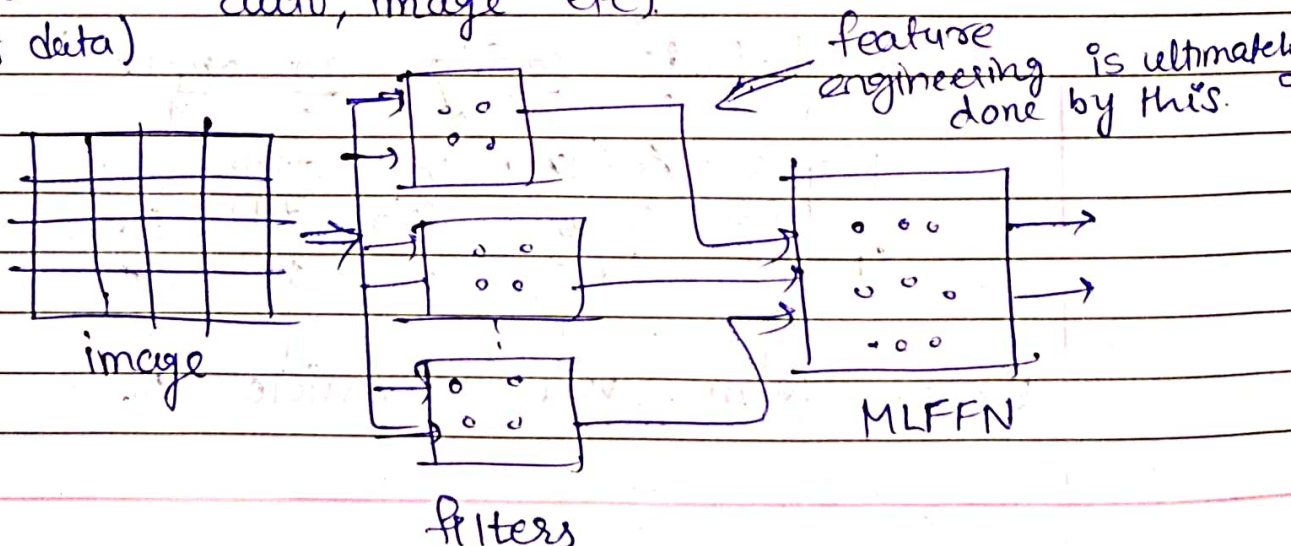
For training,

- Select bunch of inputs and then train.

instead of one at a time (Problem :

while training current symbol, it might happen that, it forgot Pass symbols).

- CNN - Does not require feature Engineering.
(works with huge amount of data) - works with any kind of data (i.e. text, auto, image etc).



cost : Computation cost will be increased as needs to ^{decide} ~~find~~ weight values huge amount of.

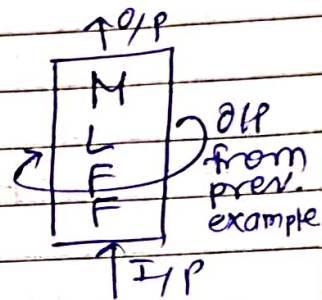
- It has more than 3 layers.

RNN (Recurrent Neural Networks)

→ Sequence Labeling Problem

Ram eats an apple.

labeled
in sequence



eg

- speech to text conversion

- Machine translation:

- translate from - Eng. to hindi

- Eng. to Gujarati etc.

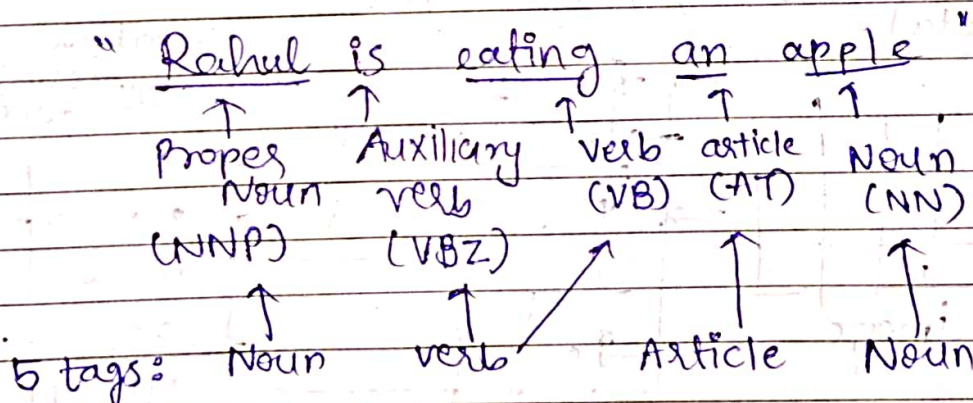
→ First model - IBM

↳ statistical Machine Translation

Nowadays,

- Neural Machine Translation (NMT) - deep learning based

→ Part of Speech Tagging



Wednesday
February
Rahul
Object
(Proper noun)

Day
Month
Boy
class
(Common noun)

→ Multilayer Feed Forward Neural Network

① No. of output layer neurons

→ # POS Tag.

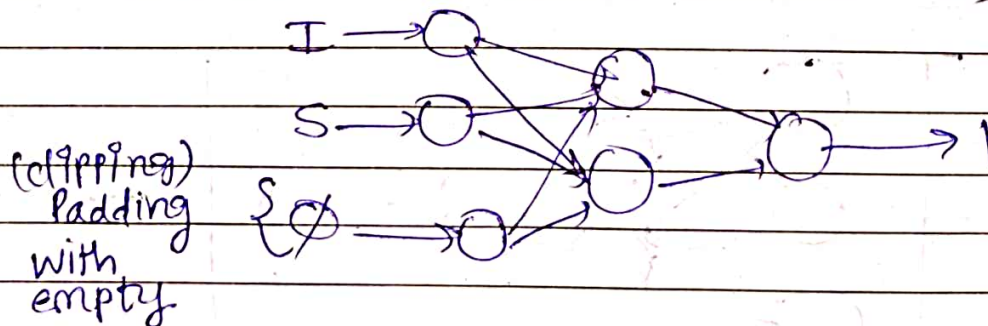
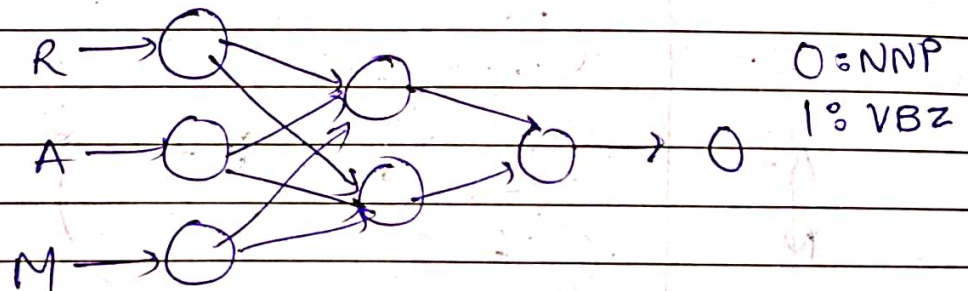
5 output classes, NNP, VBZ, VB, AT, NN

② No. of Input layer neurons

→ - No. of words in the sentence

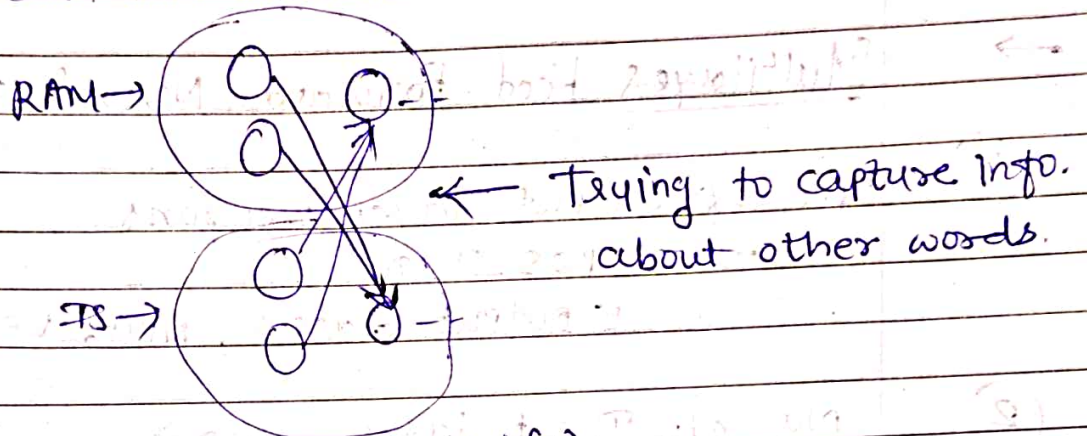
Not fixed. → Can we give one word as an input?

one char one neuron

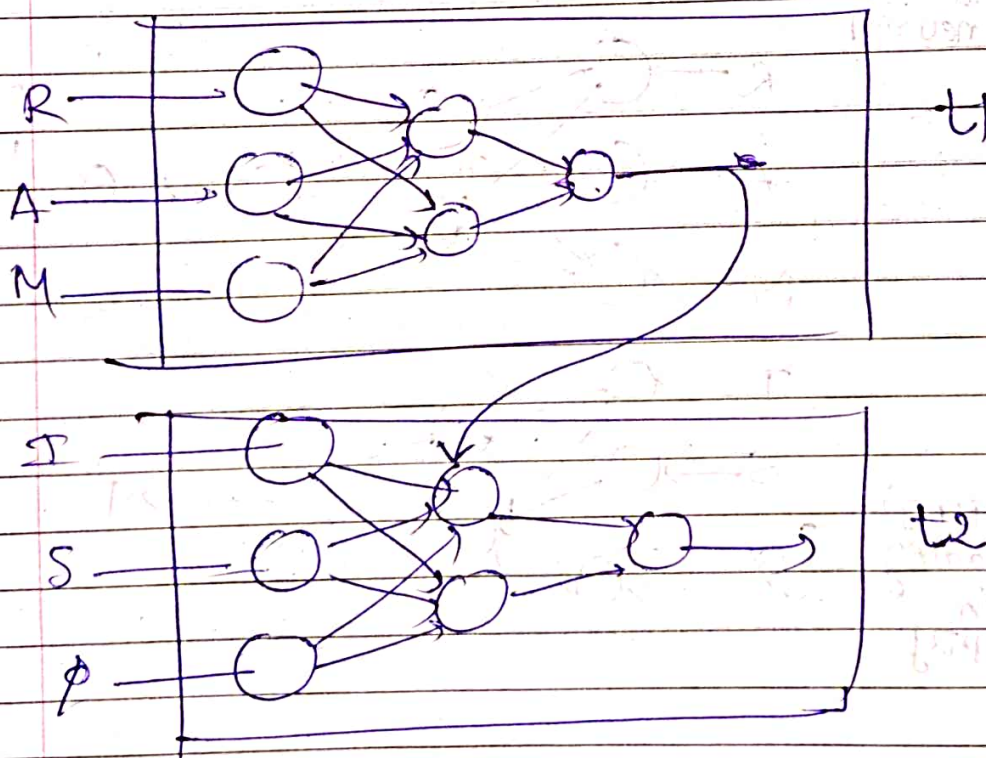


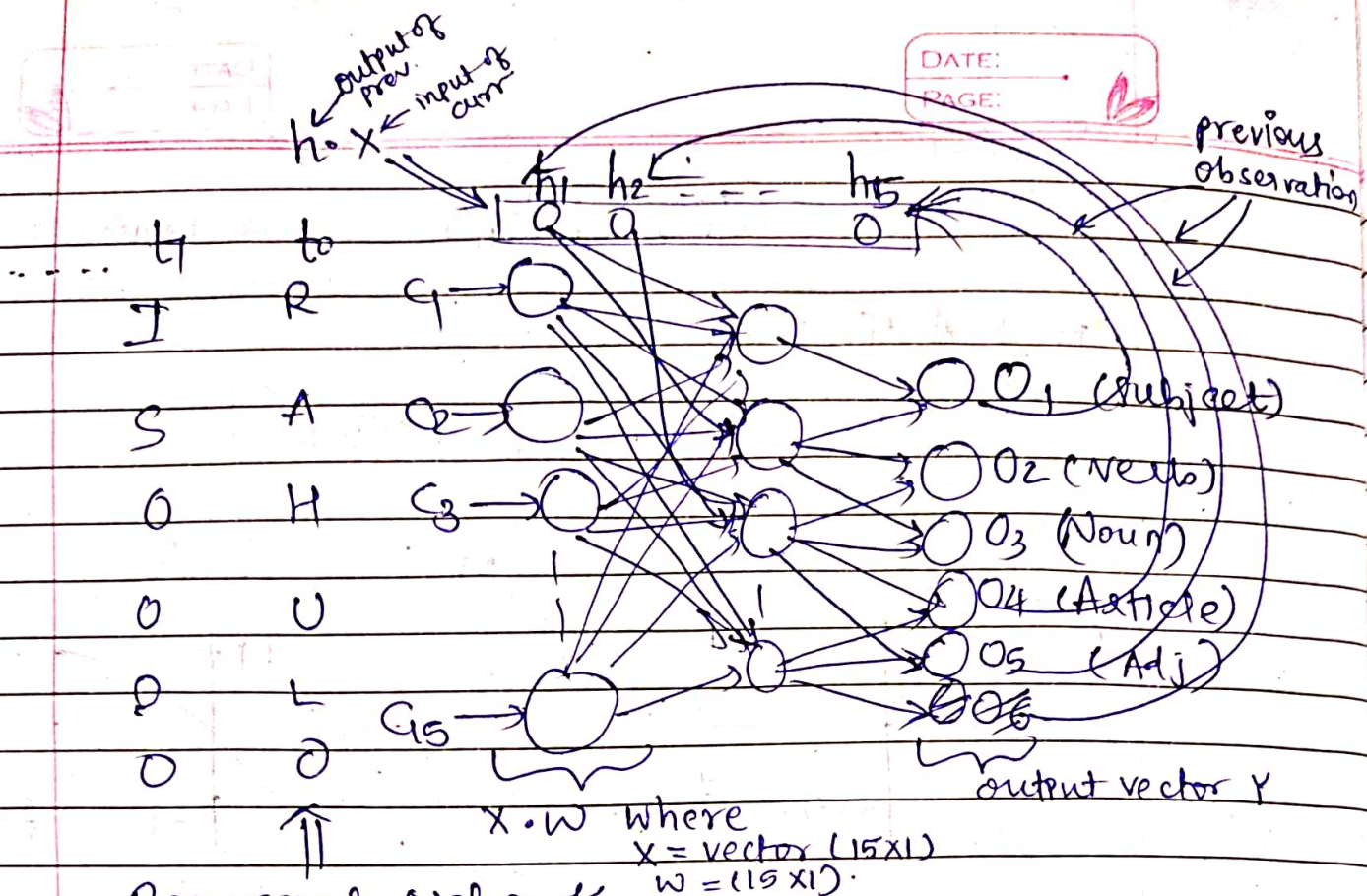
- ① Decide size of word Input
 - English follows generally SVO (Subject, ^{verb} ~~to~~ object).
 so, Ram $\rightarrow$ is.

Connect two networks



Prob: To decide $(N+1)$, ^{identify} see (N) $\leftarrow$ Ideally, but
 in this we process both together



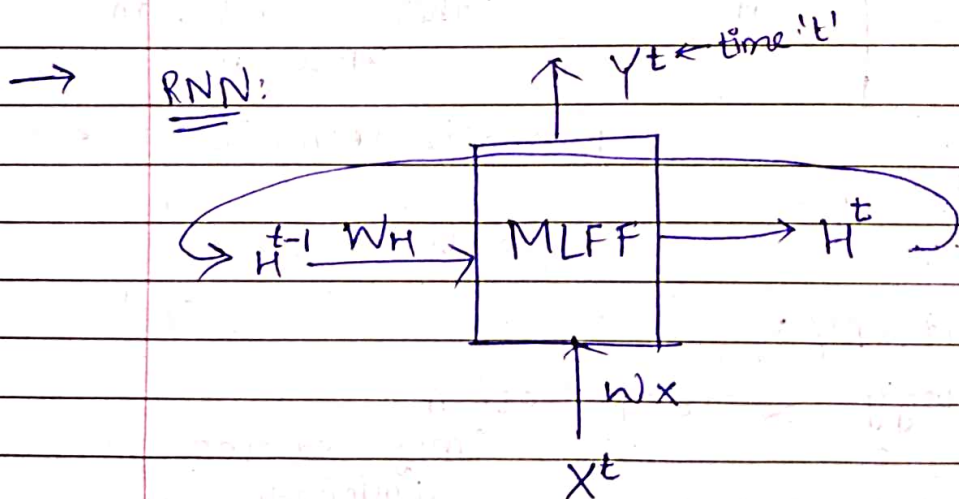


Recurrent Network

- Output previous input influences the response and subsequent weight update for the present input

→ Hidden Markov model - statistical model of sequence labeling problem.

→ CNN & MLFFNN - $n+1$ ^{input} example is not dependent on n (previous) inputs unlike RNN.



$$y^t = f(x^t, h^{t-1})$$

① Summation

$$y = f(x^t w_x + h^{t-1} w_h)$$

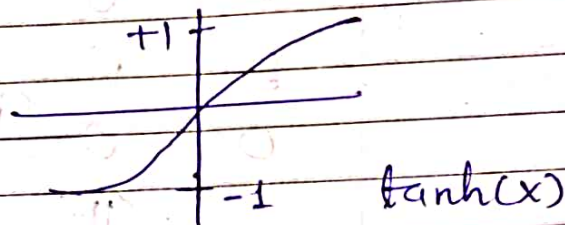
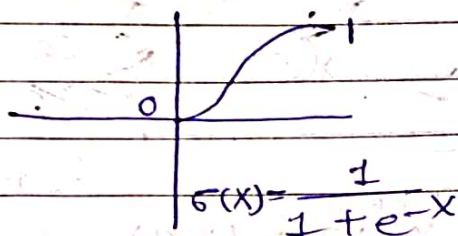
2 tensors: xw, hw

② Apply Activation func.

$$y^t = f[\tanh(x^t w_x + h^{t-1} w_h)]$$

→ logistic sigmoid vs tanh

①



② Derivation:

$$\sigma(x) (1 - \sigma(x))$$

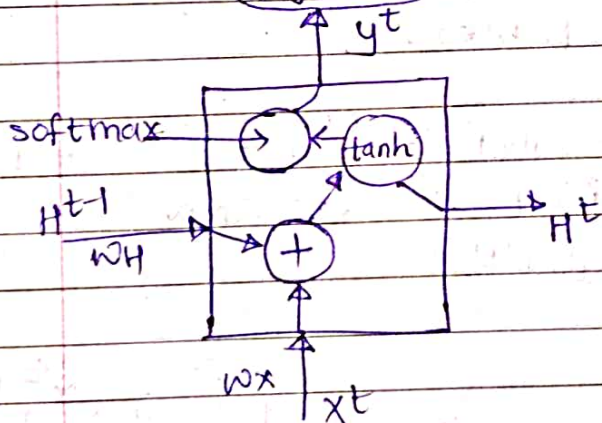
and second order

Derivation:

$$(1 - \tanh^2(x))$$

First order derivative of tanh is larger than sigmoid as the value of x increases (approx. 4 times)

log loss → E(Error)



NNP NN AT VBZ VB

0.6	0.8	0.2	-0.4	-0.3
-----	-----	-----	------	------

← applied tanh (1 to -1)

↓
this does not sum up to one.

Normalization: (which is done in softmax)

$$\text{softmax} = \frac{e^x}{\sum e^x}$$

Cross Entropy:

Max.

likelihood

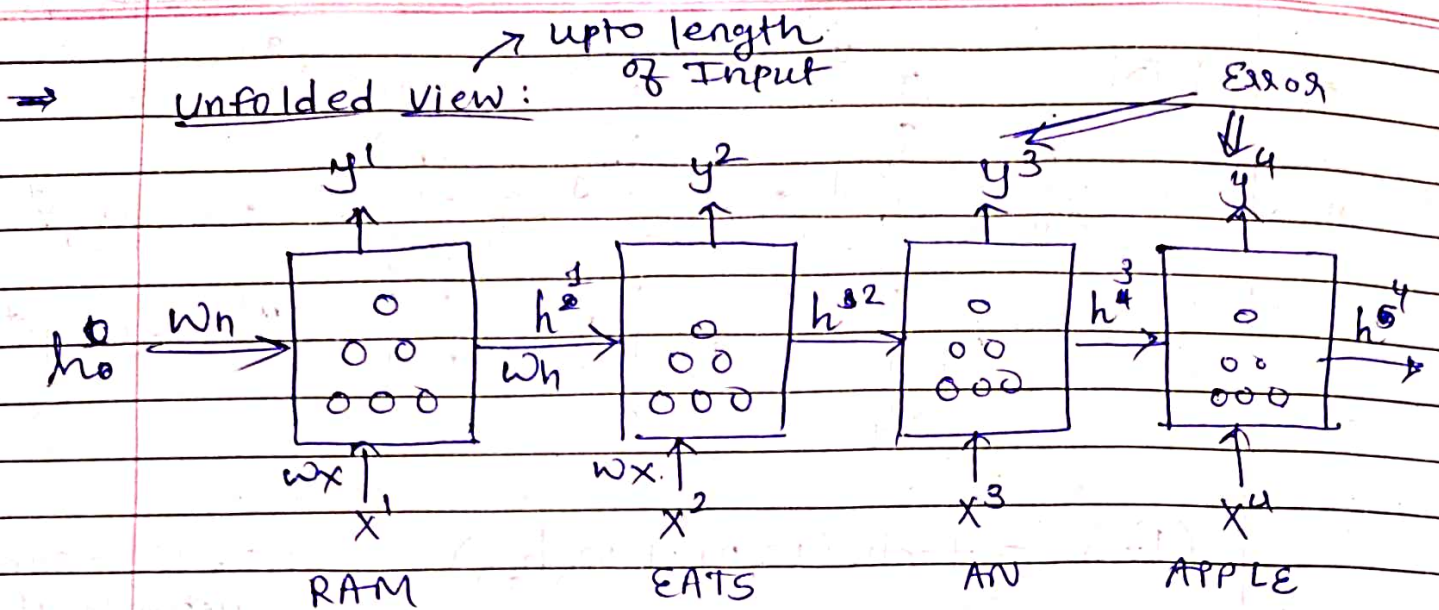
Estimation:

$$-\sum y \log y \leftarrow \text{expectation}$$

↑
predicted output

(min. Expectation)
minimize

(y should be error (Actual-predicted))



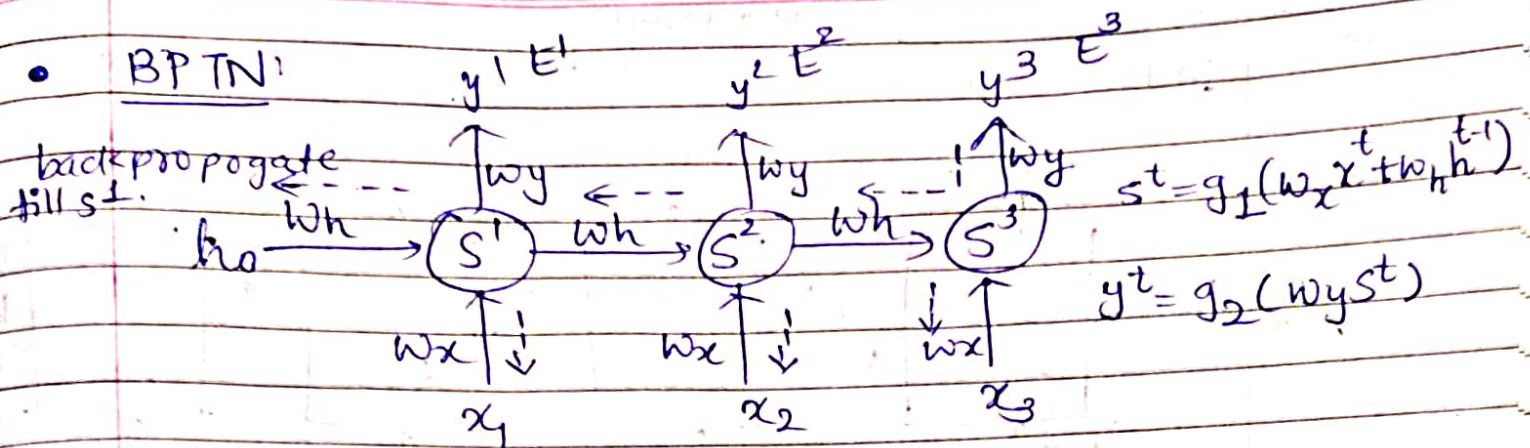
- length of $y = 5$ (5 classes to predict: NNP, NN...)
 - length of $h = 5$
 - length of $x = 5$ (5 bit vector)
- Output y and h didn't have any subscript as they are in same form and their value are different at each time step.
- error calculation happens at 4th time step as we've removed intermediate output. error calculated at end can be from any of the intermediate McFF or all previous McFF.
- update the weight n no. of time where $n = \text{no. of McFF to correct the error}$



Backpropagation through time (BPTT)

- As it propagates to previous, the value of δ ↓ (That's why 'tanh' is used as we want to propagate and pass through long network).

• BPTT:



① $h_t = \tanh(w_x x_t + w_h h_{t-1})$ s^1, s^2, s^3 - same network at diff time

② $y_t = w_y h_t$ ③ $s = \text{Softmax}(y_t)$

④ $L = -\log S_m$ (log loss)

$\Rightarrow \frac{\partial E^1}{\partial w_y} ; \frac{\partial E^1}{\partial w_x} ; \frac{\partial E^1}{\partial w_h}$ } error E^3 .

(In diagram) $\left(\frac{\partial E^1}{\partial y^1} \cdot \frac{\partial y^1}{\partial w_y} \cdot \frac{\partial w_y}{\partial s} \cdot \frac{\partial s}{\partial w_x} \right)$

Other from p.d.f

$\Rightarrow \frac{\partial E^2}{\partial w_x^2} = \left(\frac{\partial E^2}{\partial y^2} \cdot \frac{\partial y^2}{\partial w_y} \cdot \frac{\partial w_y}{\partial s^2} \cdot \frac{\partial s^2}{\partial x_2^2} \right)$

$\frac{\partial E^2}{\partial w_x^1} = \left(\frac{\partial E^2}{\partial y^2} \cdot \frac{\partial y^2}{\partial w_y} \cdot \frac{\partial w_y}{\partial s^2} \cdot \frac{\partial s^2}{\partial x_1^1} \right)$

E^2 is depend upon current & prev.

Kth layer:

$$\frac{\partial E^k}{\partial w_x} = \sum_{i=1}^k \left(\frac{\partial E^i}{\partial y^i} \cdot \frac{\partial y^i}{\partial w_y} \cdot \frac{\partial w_y}{\partial s^{k,i}} \cdot \frac{\partial s^{k,i}}{\partial x_{k,i}} \right)$$

Recursive update of weights $\rightarrow$

RNN variants :

① Output and Error at each level

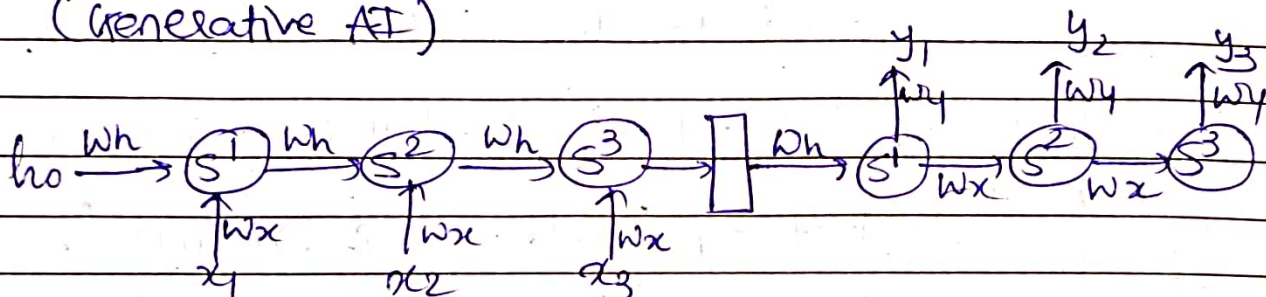
Eg POS Tag

Diagram $\Rightarrow$

② Output and Error at last level Eg Text classification
- only at y^3 & ϵ^3 . "Sentiment Analysis".

③ Output & Input at Intermediate level

④ First read Input sequence & then produce the output sequence
(Generative AI)



— For calculating error at n^{th} level, ^{weights of} at 1st level will be updated for n times.

$\hookrightarrow$ this problem is known as "Vanishing" or "Exploding gradient".

— Exploding gradient's solⁿ:

- update weights upto some limit.

$\Rightarrow$ 2) Sentiment Analysis can also be done by type ① but problem is "Input size!"
need to do feature engineering

3) used to handle vanishing gradient issue.

4) Input part & Output part ← RNN is divided into
Encoder Decoder.

 → overall learning from Input

- Also used in "Language Modeling".

↳ getting the characteristic of lang.

⇒ creates a map of tokens of the language.

⇒ each word is represented as a vector
(word vector) 'W'

⇒ vector of a word contains words which are related to the word W.

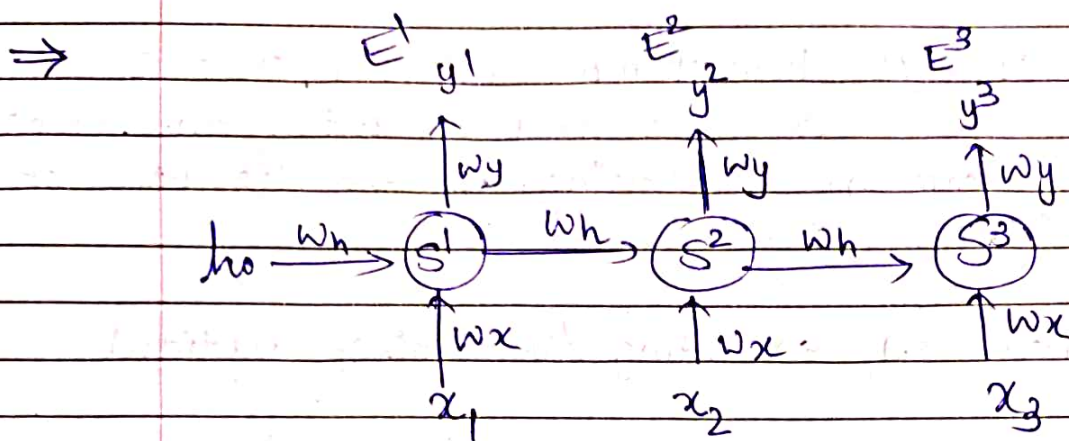
word vector { dog = [animal, domestic, stray, cat]
Bat = [ball, stumps, mammal,]

① scan through entire text. see neighbouring words and put them into word vector.

② Mask language Modeling.

- Take a paragraph, Mask some words at random. ↳ remove that word & put blank.

- Train the system to predict the masked word.



Calculate:

① $\frac{\partial \epsilon^3}{\partial w_y}$ ② $\frac{\partial \epsilon^3}{\partial w_x}$ ③ $\frac{\partial \epsilon^3}{\partial w_h}$

① $\frac{\partial \epsilon^3}{\partial w_y} = \frac{\partial \epsilon^3}{\partial y^3} \cdot \frac{\partial y^3}{\partial w_y}$

② $\frac{\partial \epsilon^3}{\partial w_x} = \frac{\partial \epsilon^3}{\partial y^3} \cdot \frac{\partial y^3}{\partial w_y} \cdot \frac{\partial w_y}{\partial s^3} \cdot \frac{\partial s^3}{\partial w_x}$

③ $\frac{\partial \epsilon^3}{\partial w_h} = \frac{\partial \epsilon^3}{\partial y^3} \cdot \frac{\partial y^3}{\partial s^3} \cdot \frac{\partial s^3}{\partial s^2} \cdot \frac{\partial s^2}{\partial w_h} + \frac{\partial \epsilon^3}{\partial y^3} \cdot \frac{\partial y^3}{\partial s^3} \cdot \frac{\partial s^3}{\partial s^2} \cdot \frac{\partial s^2}{\partial s^1} \cdot \frac{\partial s^1}{\partial w_h}$

$$= \sum_{i=1}^n \frac{\partial \epsilon^i}{\partial y^n} \cdot \frac{\partial y^n}{\partial s_n} \cdot \frac{\partial s_n}{\partial s_i} \cdot \frac{\partial s_i}{\partial w_i}$$

③ $\frac{\partial \epsilon^3}{\partial w_h} = \frac{\partial \epsilon^3}{\partial y^3} \cdot \frac{\partial y^3}{\partial s^3} \cdot \frac{\partial s^3}{\partial w_h} + \frac{\partial \epsilon^3}{\partial y^3} \cdot \frac{\partial y^3}{\partial s^3} \cdot \frac{\partial s^3}{\partial s^2} \cdot \frac{\partial s^2}{\partial w_h} + \frac{\partial \epsilon^3}{\partial y^3} \cdot \frac{\partial y^3}{\partial s^3} \cdot \frac{\partial s^3}{\partial s^2} \cdot \frac{\partial s^2}{\partial s^1} \cdot \frac{\partial s^1}{\partial w_h}$

Problem : As time stamp increases, size of each e_2^n increasing and weights can either keep on increasing in one direction or decreasing.

⇒ Problem of Exploding / Vanishing gradient

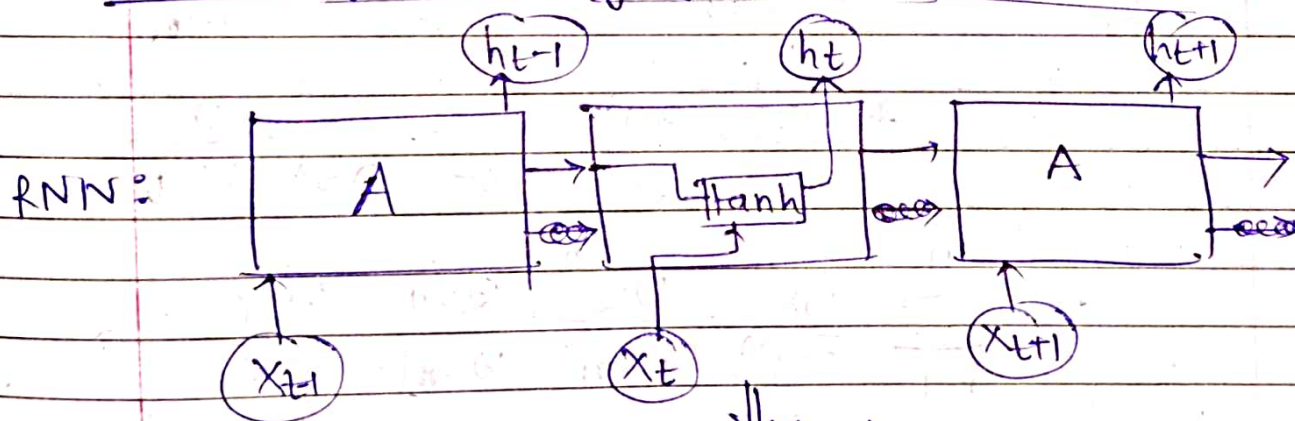
— Because of successive update of weight same weight updates many times. If that weight value is already large, it will explode

↳ Put a upper limit on weight updation.

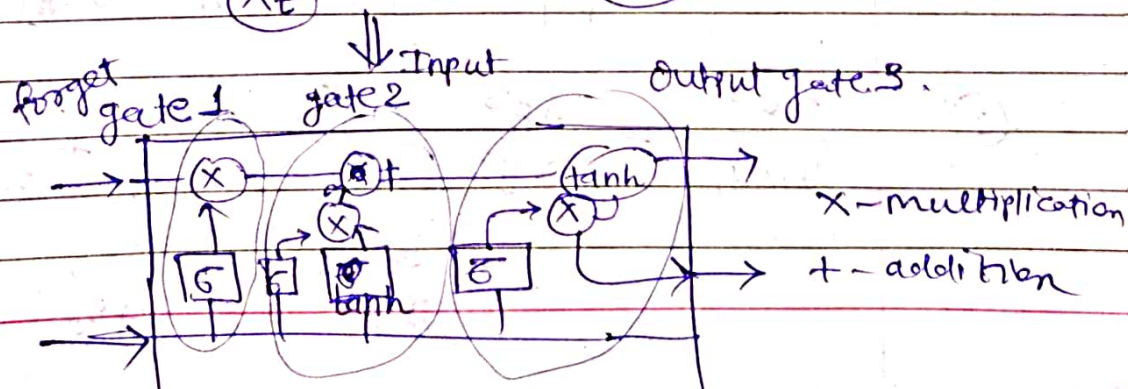
↳ weight clipping

vanishing gradient { — Because of recursion, the (partial derivative) becomes too small for earlier time stamp
↳ short term memory

⇒ Solⁿ of vanishing gradient is LSTM:



Long short term memory Network





→ Forget gate : Feed forward network with sigmoid activation

$$f_t = \overset{(0 \text{ to } 1)}{\sigma}(w_f \cdot [h_{t-1} \cdot x_t] + b_f)$$

→ Input gate

$$i_t = \sigma(w_i \cdot [h_{t-1} \cdot x_t] + b_i)$$

$$c =$$

σ is used as switch.